



PraroopAI

Praroop · the Hindi word for a draft

AI DPR / RFP Drafting & Compliance Assistant

LOW-LEVEL DESIGN

Requirements · Architecture · Agent Specifications · Interfaces

Field	Value
Document	Low-Level Design (LLD)
System	PraroopAI — AI DPR/RFP Drafting & Compliance Assistant
Programme	UKIS 2026 · ITDA, Government of Uttarakhand
Problem statement	DPR / RFP Drafting Assistant
Version	1.0
Status	Issued for review
Prepared by	Ajay Bisht — Senior Architect
Classification	Internal · non-sensitive

This document describes *how* PraroopAI is built. It assumes the reader is familiar with the problem statement but not with the implementation.

Document Control

Version	Date	Author	Summary of change
0.1	Sep 2026	A. Bisht	Initial concept and agent outline
0.5	Sep 2026	A. Bisht	Deterministic money engine; RFP and DPR separated into distinct section sets
0.9	Sep 2026	A. Bisht	Streaming progress bus; configurable logging; reviewer whitelist
1.0	Sep 2026	A. Bisht	Deterministic clause fallbacks; parallel drafting; issued for review

Intended readership

Reader	Sections of most interest
Evaluation panel	1 — 4, 8 (what it does, how it is structured, why the verdict is trustworthy)
Solution architect	4 — 11 (architecture, agents, interfaces, deployment)
Developer	5 — 10, 12 — 14 (specifications, data model, API, tests)
Operations / IT	11 — 13 (deployment, configuration, failure modes, logging)

Contents

1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions and abbreviations	5
2. Requirements	5
2.1 Problem statement	5
2.2 Functional requirements	6
2.3 Non-functional requirements	6
2.4 Constraints and assumptions	6
3. Solution Overview	7
3.1 The governing principle	7
3.2 Key design decisions	7
3.3 Why compliance is a gate and not a score	8
4. System Architecture	8
4.1 Layer responsibilities	10
4.2 Technology choices	10
5. Agent Design	10
5.1 Agent roster	10

5.2 Intake agent	11
5.3 Derivation engine	11
5.4 Drafting agent	11
5.5 Compliance agent — the independent critic	12
5.6 Reviewer agent — repair in two stages	12
5.7 Score node	12
6. Agent Flow and Routing	12
6.1 Graph definition	14
6.2 Routing predicate	14
6.3 Termination	14
7. Request Lifecycle	14
7.1 Streaming protocol	18
7.2 Concurrency model	18
8. Compliance Engine	18
8.1 Rule schema	20
8.2 Rule-pack inventory	20
8.3 Context variables	20
8.4 Scoring and the hard gate	21
8.5 Deterministic fallbacks	21
8.6 Sanitising model output	21
9. Data Model	22
9.1 Pipeline state	22
9.2 Document structure	22
9.3 Finding	23
10. Interface Specification	23
10.1 HTTP endpoints	23
10.2 Health response	23
10.3 Exported document	23
11. Deployment and Configuration	24
11.1 Runtime	26
11.2 Configuration reference	26
11.3 Provider selection	26
11.4 Performance characteristics	26
12. Error Handling and Resilience	27
13. Observability	27

13.1 Representative output	28
14. Testing Strategy	28
14.1 Adversarial end-to-end test	28
15. Limitations and Roadmap	28
15.1 Known limitations	29
15.2 Roadmap	29

1. Introduction

1.1 Purpose

PraroopAI helps a government officer produce a compliant, ready-to-issue **RFP** or **DPR**. This document specifies the internal design: the layers, the agents, the interfaces between them, and the rules that decide whether a document may be issued at all.

1.2 Scope

In scope	Out of scope
Multi-agent drafting pipeline and its routing	Procurement workflow beyond document preparation (tender publication, bid receipt, evaluation)
Deterministic compliance engine and rule-pack format	e-Signature, DSC and formal approval routing
Provider-agnostic LLM integration	Model training or fine-tuning — PraroopAI uses models as they are
Officer-facing interface and streaming protocol	Identity management; the pilot assumes an authenticated network
Gated DOCX export	Long-term document repository and versioning

1.3 Definitions and abbreviations

Term	Meaning
DPR	Detailed Project Report — the planning and appraisal document prepared before a project is sanctioned
RFP	Request for Proposal — the tender document issued to bidders
EMD	Earnest Money Deposit, also called Bid Security (GFR Rule 170)
PBG	Performance Bank Guarantee, also called Performance Security (GFR Rule 171)
LD	Liquidated Damages — the penalty for delay (GFR Rule 175)
GFR	General Financial Rules, 2017
Rule-pack	A versioned YAML file holding a group of related compliance rules
Finding	A single failed rule, carrying its identifier, severity and citation
Hard gate	The rule that export is refused while any mandatory finding is open
SSE	Server-Sent Events — the one-way stream used to push live progress

2. Requirements

2.1 Problem statement

Preparing a DPR or an RFP means satisfying dozens of mandatory rules drawn from several frameworks at once — financial norms, procurement and GFR guidelines, security obligations, service levels and penalty clauses. The rules are public and finite, but they are scattered, and applying every one of them correctly depends on individual experience. A single missing clause can cause a tender to be rejected or flagged in audit, months after the officer has moved on.

An ordinary chatbot makes this worse rather than better. It produces fluent text that *looks* compliant, offers no evidence, and miscalculates money. The design problem is therefore not "generate a document" but **generate a document whose compliance can be demonstrated**.

2.2 Functional requirements

ID	Requirement	Verified by
FR-01	Accept a plain-language project description and structure it into a project brief	\$5.2.1
FR-02	Support two distinct document types, RFP and DPR, with different section sets	\$5.1, \$9.2
FR-03	Derive every statutory figure (EMD, PBG, LD cap, milestone amounts) from the officer's inputs	\$5.2.2
FR-04	Draft each section of the document using a language model	\$5.2.3
FR-05	Audit the draft against every applicable rule and report each failure with its statutory citation	\$8
FR-06	Repair failing sections automatically and re-audit	\$5.2.5
FR-07	Refuse export while any mandatory finding is open	\$8.4
FR-08	Stream progress to the officer while the run is in flight	\$7.1
FR-09	Export a formatted DOCX including a compliance scorecard and the statutory basis	\$10.1
FR-10	Allow the LLM provider to be changed by configuration alone	\$11.2

2.3 Non-functional requirements

ID	Requirement	Design response
NFR-01	Correctness of figures. No monetary value in the output may be invented or mis-scaled by the model	Money is computed in Python; model output is rewritten to canonical figures (§8.6)
NFR-02	Trustworthy verdict. A compliance result must not depend on model judgement	The critic is a deterministic evaluator over YAML rules (§8)
NFR-03	Data residency. Project data must be able to remain inside the government network	On-premise model by default; hosted providers are opt-in (§11)
NFR-04	Transparency. An operator must be able to see what the system is doing and why	Four log levels including full prompt capture (§13)
NFR-05	Responsiveness. A long run must never appear frozen	Per-section progress events and a heartbeat (§7.1)
NFR-06	Portability. Deployable without a build toolchain	Single Python process serves both API and UI (§11)
NFR-07	Maintainability. Rules must be changeable by a domain expert, not only a developer	Rules and statutory ratios live in versioned YAML (§8.1)

2.4 Constraints and assumptions

#	Statement	Type
C-1	The knowledge base is a bounded set of public rules, not a corpus of historical documents	Constraint

#	Statement	Type
C-2	The deployment target may have no GPU; the design must degrade in speed, not in correctness	Constraint
C-3	Small models (1.5B class) follow instructions unreliably and must be assumed to misbehave	Assumption
C-4	The officer remains the author of record; PraroopAI prepares a draft and evidence, it does not approve	Assumption
C-5	The network is already authenticated; user identity is out of scope for the pilot	Assumption

3. Solution Overview

3.1 The governing principle

The language model writes prose. Code owns the facts.

Every significant design decision follows from that sentence. A model is good at turning structured facts into formal prose and poor at arithmetic, at following rigid formats, and at telling you honestly whether it has complied. So the system gives it the first job and withholds the other three.

Concern	Owner	Guarantee it provides
Section prose	Language model	Readable, project-specific text
Every monetary figure	backend/core/derivations.py	Amounts are computed; any amount the model writes is rewritten to the canonical value
Compliance verdict	backend/core/rule_engine.py	A stated percentage is a fact, reproducible without a model
Formulaic clauses	agents/fallbacks.py	A deterministic rule always has a deterministic repair, so the gate can always be closed honestly
Document structure	agents/doc_structure.py	An RFP and a DPR are different documents, not one template renamed

3.2 Key design decisions

Decision	Alternative considered	Why this choice
Bounded rule-packs rather than retrieval over a document corpus	RAG over past tenders	The governing rules are public and finite. Encoding them is faster, fully explainable, and avoids a large curation effort
Deterministic compliance critic	Ask a second model to review the first	A reviewing model can be wrong in the same direction as the writer. Code cannot be persuaded
One LLM call per section	One call producing the whole document	Small models degrade sharply as output length grows; narrow tasks produce markedly better sections
Deterministic clause fallback	Loop until the model succeeds	With a weak model the loop never terminates and the gate stays shut forever. See §5.2.5
Graph executed on a worker thread	Run inline in the request handler	A node only returns when it finishes; drafting would be silent for minutes. See §7.2
Provider-agnostic REST client	Vendor SDKs	Keeps the dependency surface small and makes provider choice a configuration value

3.3 Why compliance is a gate and not a score

A score invites negotiation; a gate does not. If the system merely reported "94% compliant", the officer would be left to decide whether the missing 6% mattered — which is precisely the judgement the tool exists to support. Instead, findings carry a severity. Any finding marked **mandatory** blocks export, and the API returns 423 Locked rather than a document. Recommended findings are reported but do not block.

This is only defensible because the verdict is deterministic. A gate driven by model opinion would be an obstacle; a gate driven by an evaluator over published rules is a control.

4. System Architecture

PraroopAI is a single Python process exposing a small HTTP surface. Internally it is arranged in six layers; each depends only on the layer beneath it, and the domain core depends on nothing above it.

PraroopAI — System Architecture

Layered view. Each layer depends only on the layer beneath it.

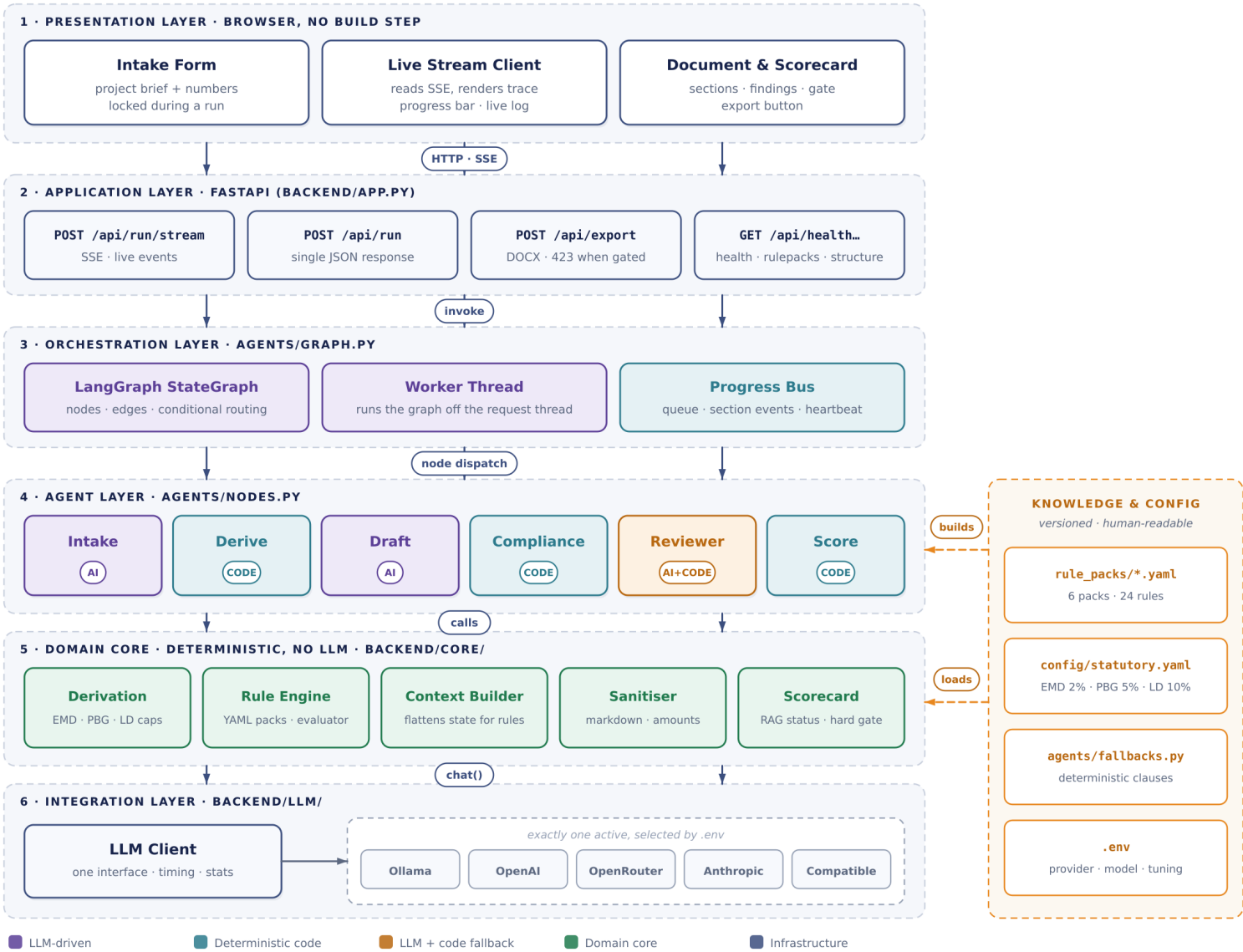


Figure 1 — Layered system architecture.

4.1 Layer responsibilities

Layer	Module	Responsibility	Depends on
1 • Presentation	frontend/	Collect inputs, consume the event stream, render the document, scorecard and gate	Layer 2 over HTTP
2 • Application	backend/app.py	Route HTTP requests, serialise the event stream, enforce the export gate at the boundary	Layer 3
3 • Orchestration	agents/graph.py	Compile and execute the state graph; run it on a worker thread; drain the progress queue	Layer 4
4 • Agent	agents/nodes.py	The six pipeline steps and the routing predicate between them	Layers 5 and 6
5 • Domain core	backend/core/	Derivation, rule evaluation, context construction, sanitising, scoring. No LLM, no I/O	Nothing
6 • Integration	backend/llm/	One chat interface; provider selection, timing and call statistics	External service

4.2 Technology choices

Concern	Choice	Note
Web framework	FastAPI + uvicorn	Typed request models; native streaming responses
Agent orchestration	LangGraph	Explicit nodes, edges and conditional routing; an equivalent in-process runner exists as a fallback if the package is absent
Rule evaluation	Restricted AST evaluator	A whitelist of comparison and boolean operators. <code>eval()</code> is never used on rule text
Model access	httpx against native REST	No vendor SDKs; five providers behind one interface
Document export	python-docx	Headings, bullets, scorecard table and statutory references
Interface	Hand-written HTML, CSS and JavaScript	No build step, which matters for an air-gapped install

5. Agent Design

The pipeline has six nodes. Three are driven by the language model, two are pure code, and one — the reviewer — attempts the model first and falls back to code. Every node takes the shared state and returns a partial update; none mutates the state of another.

5.1 Agent roster

Node	Kind	Consumes	Produces	Principal risk
Intake	LLM	Free text + form inputs	Structured brief	Malformed JSON — falls back to form inputs
Derive	Code	Brief	Statutory figures + canonical money strings	None; pure function
Draft	LLM	Brief + derived figures	One body per section	Thin or empty sections — caught by the critic

Node	Kind	Consumes	Produces	Principal risk
Compliance	Code	Brief, derived, sections	Findings	None; pure function
Reviewer	LLM + Code	Findings + sections	Repaired sections	Model fails to comply — deterministic fallback engages
Score	Code	Findings	Scorecard + gate decision	None; pure function

5.2 Intake agent

Aspect	Specification
Purpose	Convert a sentence an officer would actually type into a structured brief the rest of the pipeline can rely on
Input	Free-text request; explicit form fields (cost, duration, milestones, storage)
Processing	Single LLM call requesting strict JSON. Explicit form fields then overwrite anything the model inferred, because a typed number is more trustworthy than an extracted one
Output	brief — doc_type, title, project_cost, duration_months, funding_source, storage, data_retention_days, milestones[]
Failure handling	If the model returns unparseable output or is unreachable, the node logs a warning and proceeds on form inputs alone. Intake failure never aborts a run
Post-condition	Milestones always exist; if unspecified they are generated to sum to exactly 100%

5.3 Derivation engine

Not a language task at all. Given the project cost it computes every statutory figure from the ratios in config/statutory.yaml, and emits each one as a pre-formatted string in Indian notation.

```
EMD = 2% of project cost → ₹4,00,000 (₹4.00 lakh)
PBG = 5% of project cost → ₹10,00,000 (₹10.00 lakh)
LD cap = 10% of project cost → ₹20,00,000 (₹20.00 lakh)
Tender = Open Tender above ₹50 lakh
Milestones: 40 / 30 / 30 by default, always summing to 100%
```

These strings are handed to the drafting prompt with an instruction to copy them verbatim. The model is never asked to compute or convert an amount, and §8.6 describes what happens when it does so anyway.

5.4 Drafting agent

Aspect	Specification
Purpose	Produce the body text of each section
Strategy	One focused LLM call per section rather than one call for the whole document. A narrow task produces markedly better output from a small model, and a failure is contained to one section
Concurrency	Sections are independent, so up to DRAFT_CONCURRENCY run at once. Completion order is irrelevant; sections are reordered into canonical document order before scoring
Retry	A section shorter than MIN_SECTION_CHARS is requested once more with an instruction to expand
Post-processing	Every body passes through the sanitiser: markdown headings and emphasis removed, currency normalised, amounts rewritten to canonical figures

Aspect	Specification
Failure handling	An empty section is recorded as a warning and left empty. It is not an error — the critic will raise it, and the reviewer will supply it

5.5 Compliance agent — the independent critic

The critic does not read the draft with a model. It builds a flat context from the pipeline state and evaluates every applicable rule against it. This is the node that makes the product's central claim defensible, and it is deliberately the least clever thing in the system. §8 specifies it in full.

5.6 Reviewer agent — repair in two stages

The reviewer is the only node that mixes model and code, because it is the only node that must succeed.

Stage	Behaviour
1 • Ask the model	Send the open mandatory findings and request rewritten sections. Critically, a whitelist is computed from those findings: only sections that an open finding maps to may be replaced. A key the model returns for any other section is rejected and logged
2 • Re-evaluate	Run the rules again over the amended sections
3 • Build in code	Any mandatory rule still failing that corresponds to a formulaic clause is satisfied by generating that clause from the statutory configuration
4 • Attribute	The trace records each repair as <i>(AI)</i> or <i>(rule-built)</i> , so the officer can see which parts were model-authored

Why the whitelist exists. In observed runs a small model, asked to fix two penalty findings, also returned an unrelated `sla` key containing nonsense — and the reviewer accepted it, destroying a section that had already passed. Repairs are now strictly scoped to the sections that actually failed.

Why the fallback exists. A penalty clause is a formula, not a creative task: a rate, a cap, a grace period and a per-milestone breakdown. A 1.5B model frequently cannot express it in a form that satisfies the rule, so the loop would exhaust and the gate would stay shut permanently. Since the rule is deterministic, the repair is too.

5.7 Score node

Aggregates findings into a per-framework scorecard, re-orders sections into canonical document order, and sets `can_finalize` — the single flag the export endpoint consults.

6. Agent Flow and Routing

The pipeline is a linear chain with exactly one cycle. That cycle — compliance to reviewer and back — is the whole point of the design: it is where the system inspects its own work.

PraroopAI — Agent Pipeline Flow

A linear chain with a single feedback loop: compliance ⇄ reviewer.

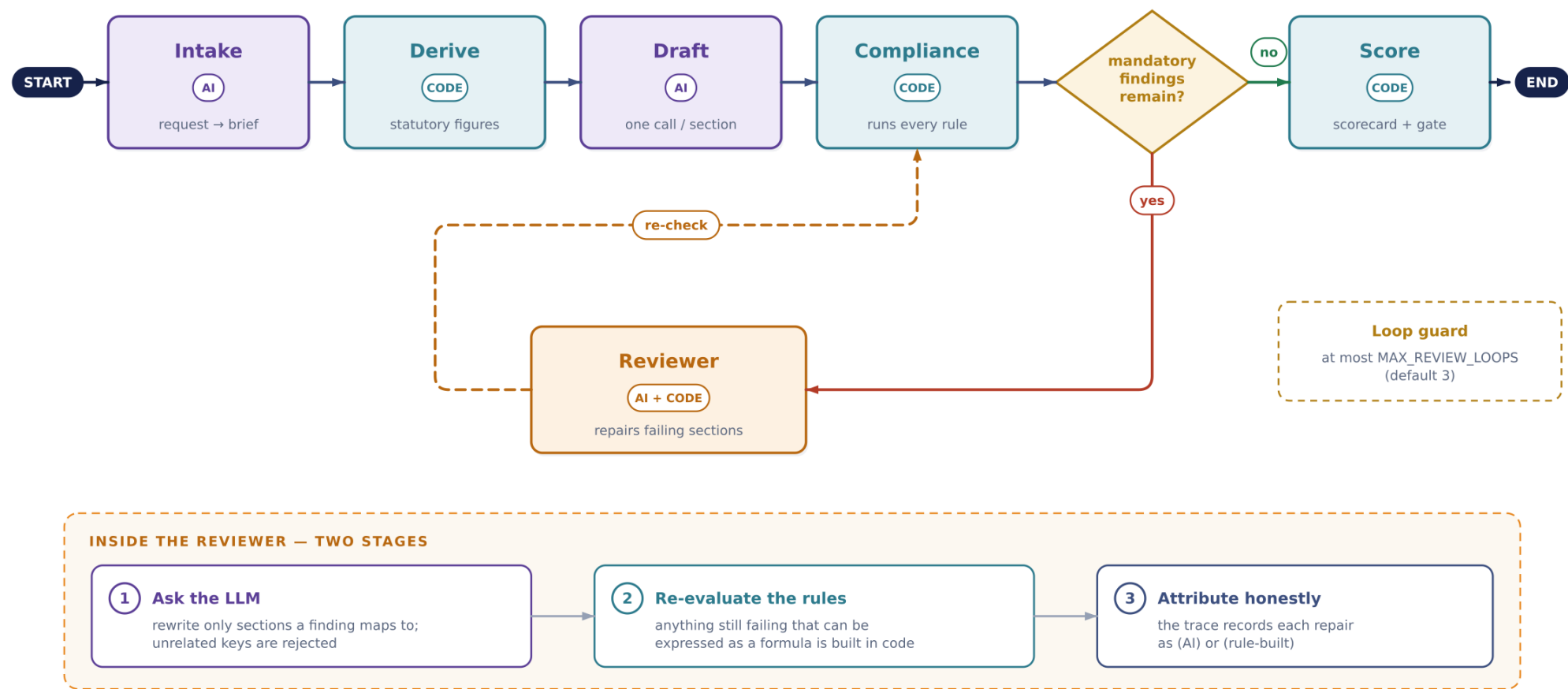


Figure 2 — Agent pipeline flow. The only cycle is compliance ⇄ reviewer.

6.1 Graph definition

```
g.set_entry_point("intake")
g.add_edge("intake", "derive")
g.add_edge("derive", "draft")
g.add_edge("draft", "compliance")
g.add_conditional_edges("compliance", route_after_compliance,
                        {"reviewer": "reviewer", "end": "score"})
g.add_edge("reviewer", "compliance") # the loop
g.add_edge("score", END)
```

6.2 Routing predicate

A single function decides whether to loop. It is deliberately trivial and has no access to a model.

```
def route_after_compliance(state) -> str:
    mandatory = count(f for f in state["findings"]
                      if f["severity"] == "mandatory")
    loops = state.get("loop_count", 0)
    if mandatory > 0 and loops < MAX_REVIEW_LOOPS:
        return "reviewer" # repair, then re-check
    return "end" # clean, or the guard has tripped
```

If the guard trips with findings still open the run does not fail — it completes honestly with a blocked gate, a warning in the log, and the outstanding findings shown to the officer. Silent success would be the worse outcome.

6.3 Termination

Property	Argument
The loop always terminates	loop_count increments on every reviewer pass and is compared against a fixed bound; no branch decrements it
The gate can always be closed honestly	Every mandatory rule either has a deterministic builder (§8.5) or depends on an officer input that cannot be synthesised — in which case the gate correctly stays shut
No node can corrupt another's work	Nodes return partial updates that are merged; the reviewer is additionally restricted by the whitelist

7. Request Lifecycle

A drafting run makes between nine and twenty-six model calls and can take minutes on modest hardware. The transport design follows from that single fact: the officer must be able to see progress at all times.

PraroopAI — Request Lifecycle (1 of 2): drafting

Browser to model and back. Dashed arrows are returns.

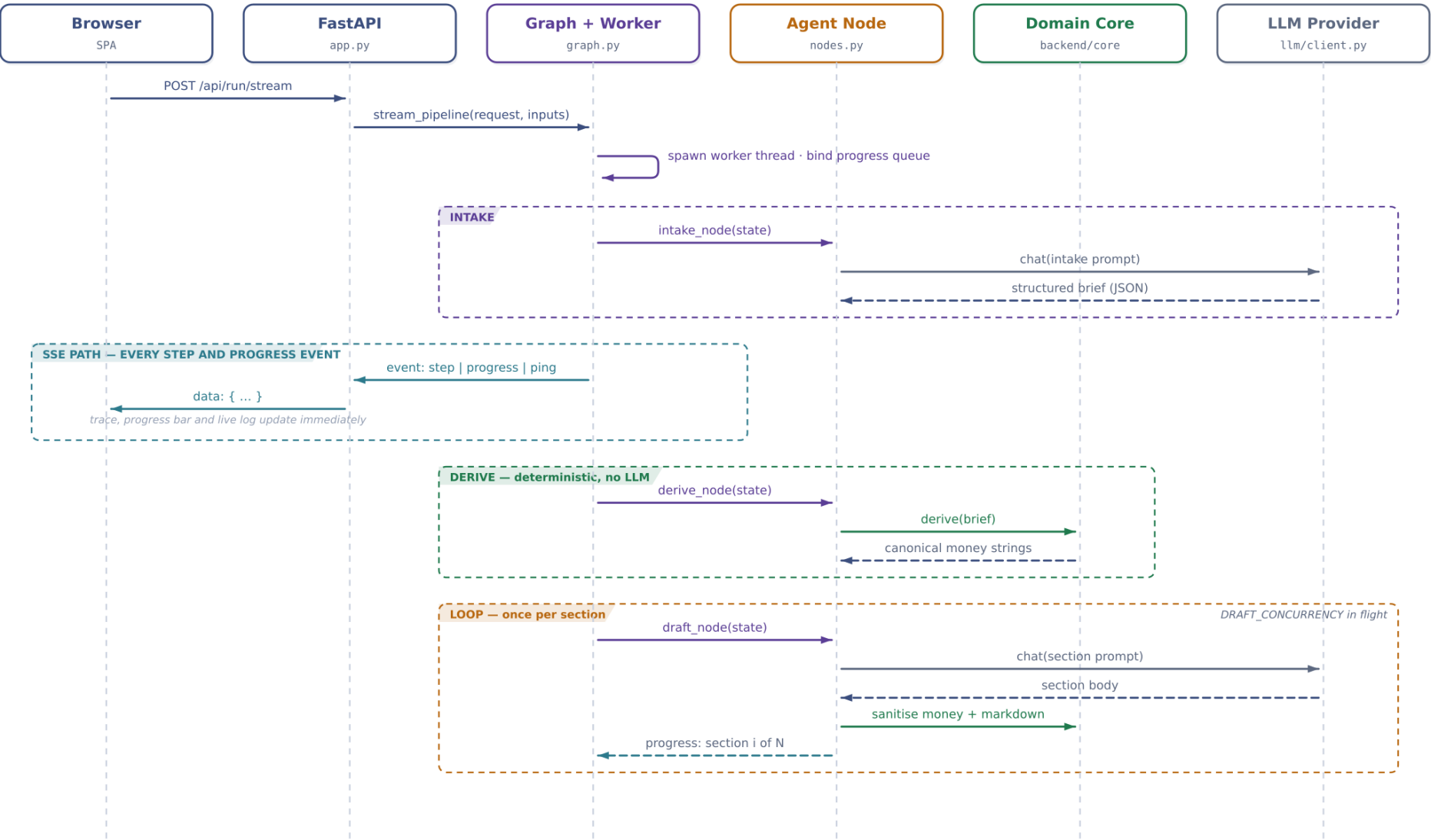


Figure 3 — Request lifecycle, part 1: setup, intake, derivation and drafting.

PraroopAI — Request Lifecycle (2 of 2): compliance and export

The critic is deterministic; so is the repair when the model cannot comply.

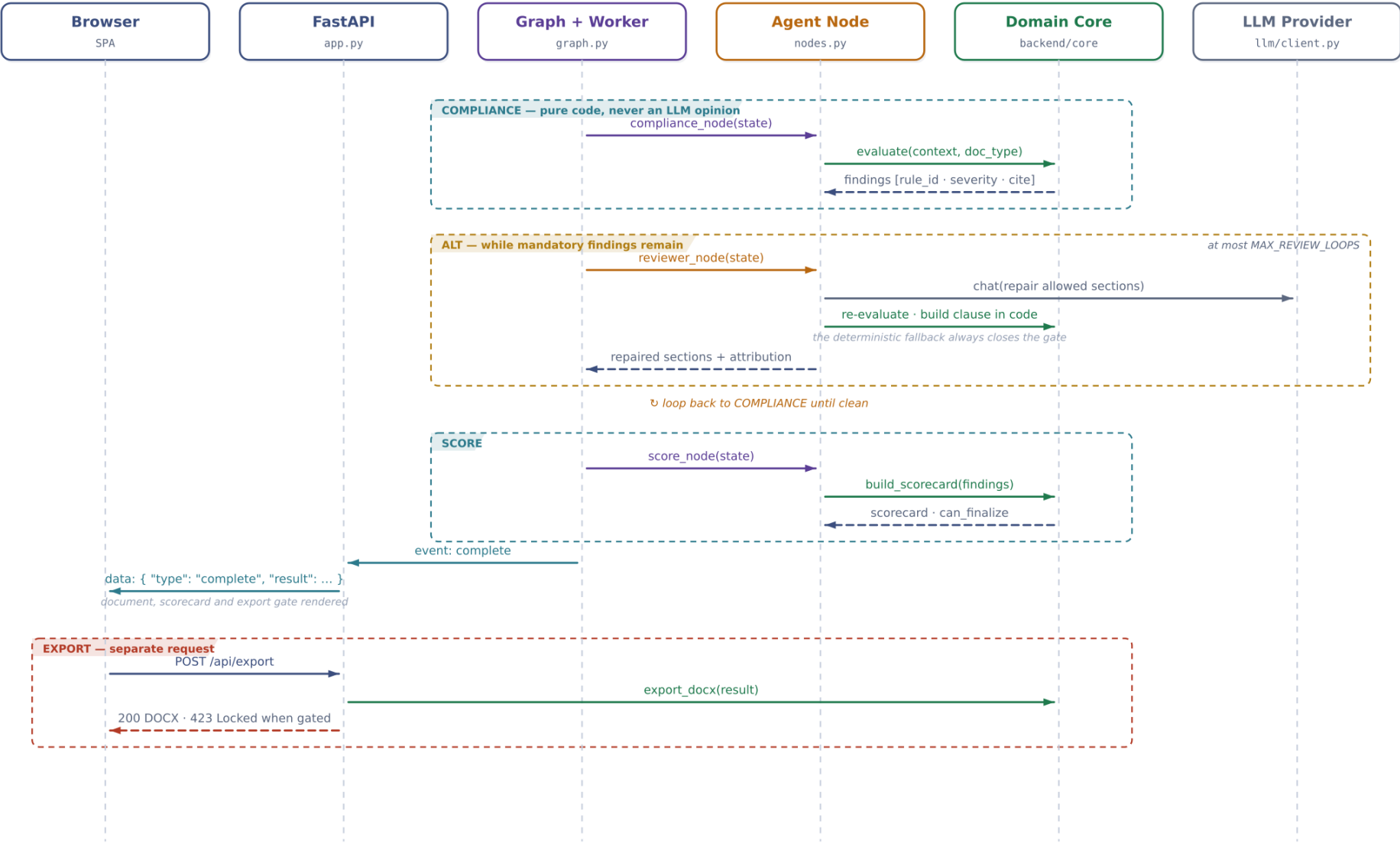


Figure 4 — Request lifecycle, part 2: compliance, repair, scoring and export.

7.1 Streaming protocol

The browser posts once to `/api/run/stream` and then reads a Server-Sent Events stream until it closes. Five event types are defined.

Event	Payload	Effect in the interface
step	One trace entry: agent, kind, detail	Appends a row to the agent trace
node	Name of the graph node that just completed	Advances the stepper
progress	stage, current, total, detail	Moves the progress bar and appends a line to the live log
ping	Elapsed seconds	Emitted every second while a node is busy, so the connection is never idle and the officer sees the run is alive
complete	Full result: sections, findings, scorecard, trace	Renders the document, scorecard and export gate

A non-streaming `/api/run` returning a single JSON document is also provided, for scripted use and testing.

7.2 Concurrency model

A LangGraph node returns its update only when it finishes. Drafting makes one call per section, so a naïve implementation emits nothing for the entire drafting phase — which in an early build appeared to the user as a frozen application.

Mechanism	Purpose
Worker thread	The graph runs off the request thread, so the HTTP handler stays free to serialise and flush events
Progress queue	A thread-local queue that any node may publish to while still executing
Queue re-binding	Drafting fans out to a thread pool; each worker re-binds the same queue, otherwise progress from those threads would be silently dropped
Heartbeat	If the queue is idle for one second the handler emits a ping, keeping intermediaries and the browser connection alive
Ordered assembly	Sections may complete out of order; they are re-ordered into canonical document order in the score node

8. Compliance Engine

This is the component the product's claim rests on, so it is specified in more detail than the rest.

PraroopAI — Compliance Engine and the Hard Gate

Rules are evaluated in code. A verdict of 100% is a fact, not a model opinion.

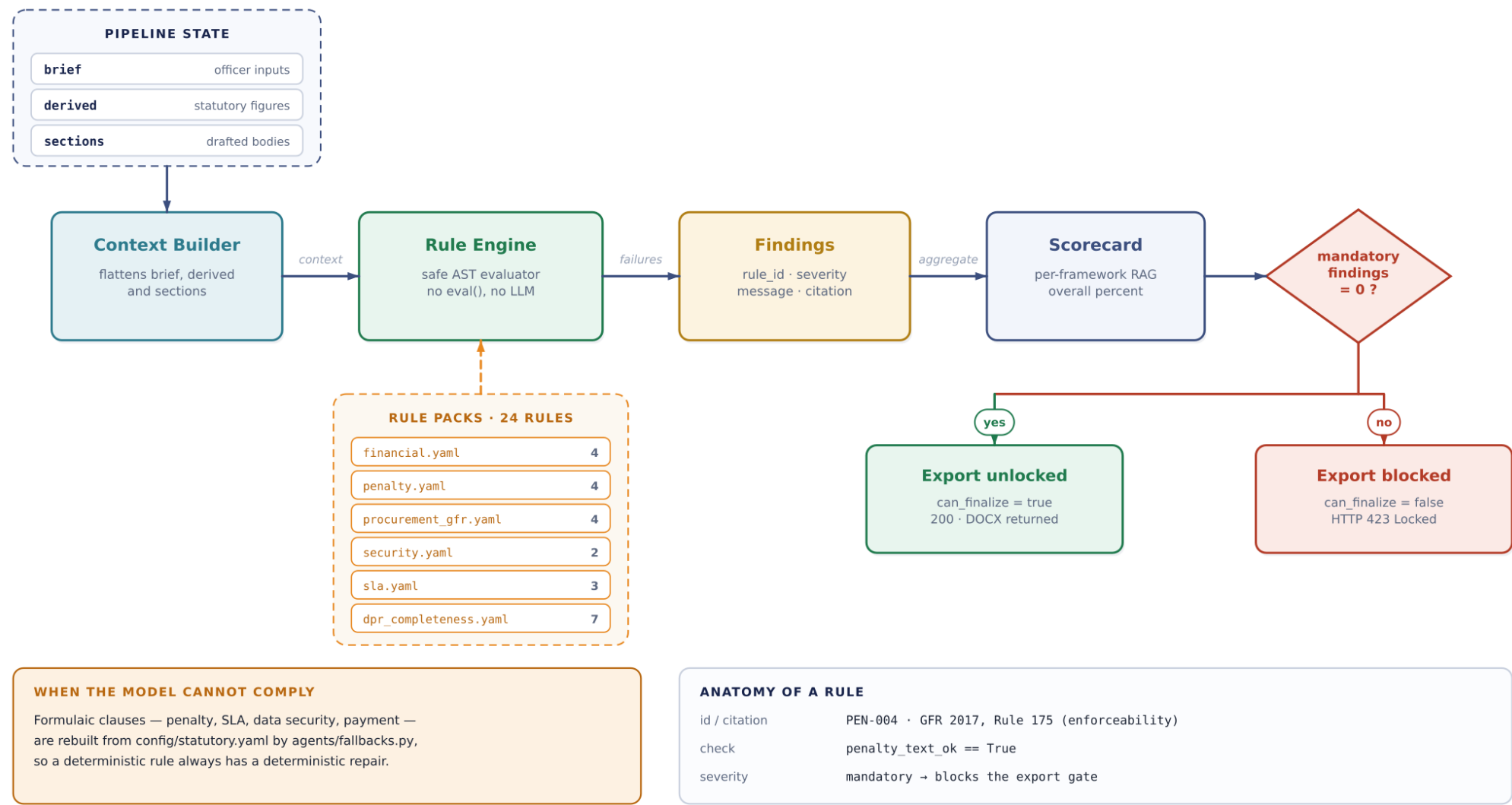


Figure 5 — Compliance engine: from pipeline state to the export gate.

8.1 Rule schema

A rule is a small, human-readable record. A domain expert can add one without touching Python.

```
- id: PEN-004
  title: "Penalty clause is substantive"
  citation: "GFR 2017, Rule 175 (enforceability)"
  applies_to: [RFP, DPR]
  check: "penalty_text_ok == True"
  severity: mandatory # mandatory blocks export; recommended does not
  message: "The penalty clause must state a rate, a cap, and cover each milestone."
  clause: "To be enforceable the LD clause must specify rate, cap, and apply per milestone."
  fix_hint: "Rewrite with LD rate, cap in rupees, grace period, per-milestone penalty."
```

Field	Purpose
id	Stable identifier. Findings cite it, so identifiers are never reused
citation	The statutory provision. Carried into the finding and into the exported document
applies_to	Document types the rule governs — an RFP and a DPR do not share every rule
check	A boolean expression over the flat context, evaluated by a restricted AST walker
severity	mandatory participates in the hard gate; recommended is reported only
message / fix_hint	Shown to the officer; the hint is also given to the reviewer agent

8.2 Rule-pack inventory

Pack	Rules	Authority	Representative rule
financial	4	GFR 2017, Rules 170 / 171 / 176	EMD is 2-3% of estimated cost
penalty	4	GFR 2017, Rule 175 + CVC	LD capped at 10%; a penalty defined per milestone
procurement_gfr	4	GFR 2017, Rules 151 / 161 / 173	Open tender above ₹50 lakh
security	2	IT Act 2000 s.43A; MeitY	Data security and retention clause present
sla	3	MeitY Model RFP	Uptime commitment of at least 99%
dpr_completeness	7	MoSPI DPR guidelines; State appraisal checklist	Risk analysis and expected outcomes present

8.3 Context variables

The context builder flattens the pipeline state into the variables rules may reference. Keeping this surface small and explicit is what allows the evaluator to stay restricted.

Group	Variables
Numeric	project_cost, emd_percent, pbg_percent, ld_percent, uptime_percent, grace_period_days
Derived boolean	milestone_payments_sum_ok, every_milestone_has_penalty, tender_mode_ok, data_residency_ok
Section presence	has_section_scope, has_section_eligibility, has_section_sla, has_section_data_security, has_section_risk_analysis, ...

Group	Variables
Quality	penalty_text_ok — true only when the clause states a rate, a cap and covers each milestone

penalty_text_ok deserves comment. An earlier version merely checked that the words "each milestone" appeared, and a model duly produced the sentence "Each milestone has been delayed by an additional 7 days due to the penalty clause" — which passed while meaning nothing. The check now requires a rate, a cap and per-milestone coverage together.

8.4 Scoring and the hard gate

Quantity	Definition
Per-framework score	passed ÷ applicable, for each rule-pack
Status	red if any mandatory rule failed; amber if only recommended failed; green otherwise
Overall percent	(total applicable – total failed) ÷ total applicable
can_finalize	true if and only if no mandatory finding is open

The gate is enforced at the boundary, not in the interface. POST /api/export re-reads can_finalize and returns 423 Locked if it is false, so disabling the button in the browser is a convenience rather than the control.

8.5 Deterministic fallbacks

Four sections are formulae rather than compositions. When the reviewer model cannot satisfy their rules, the clause is generated from the same configuration the critic checks against.

Failing rules	Section rebuilt	Generated from
PEN-001, PEN-002, PEN-004	penalty	LD weekly rate, cap percentage and amount, grace period, milestone list
SLA-001, SLA-002, SLA-003, DPR-007	sla	Uptime target, severity-banded response and resolution times, service credits
SEC-001, SEC-002	data_security	Storage location, retention days, access control, audit rights, IT Act reference
FIN-003	payment	Milestone names, percentages and amounts, summing to the contract value

Rules that depend on an officer input — for example FIN-004, which requires a positive project cost — have deliberately no fallback. There is nothing to synthesise, and the gate correctly stays shut.

8.6 Sanitising model output

Applied to every body the model returns, before the critic sees it.

Stage	Problem addressed	Behaviour
Structure coercion	The model returns a dict or list instead of prose	Flattened into labelled bullet lines
Heading strip	Echoed markdown headings such as <code>### 5. Title and inner #### Overview:</code>	All heading lines removed; the document supplies its own
Emphasis strip	Literal <code>**bold**</code> markers appearing in the text	Markers removed at ingestion so the UI and the DOCX agree

Stage	Problem addressed	Behaviour
Currency normalisation	Dollar signs and the word "dollars"	Rewritten to rupees
Amount correction	Magnitude errors, e.g. "₹200 crore" on a ₹2 crore project	Every amount is matched against the canonical set and rewritten. A power-of-ten slip is corrected; an amount matching nothing canonical is left untouched, because the sanitiser corrects, it never invents

9. Data Model

9.1 Pipeline state

A single typed dictionary threaded through the graph. `trace` uses an additive reducer so nodes append rather than overwrite.

Key	Type	Written by	Meaning
<code>request</code>	<code>str</code>	caller	The officer's free-text description
<code>raw_input</code>	<code>dict</code>	caller	Explicit form fields
<code>brief</code>	<code>dict</code>	intake	Structured project brief
<code>derived</code>	<code>dict</code>	derive	Statutory figures and canonical money strings
<code>sections</code>	<code>dict</code>	draft, reviewer, score	<code>key → {title, body}</code>
<code>findings</code>	<code>list</code>	compliance	Failed rules from the most recent audit
<code>scorecard</code>	<code>dict</code>	score	Per-framework rows, overall, <code>can_finalize</code>
<code>loop_count</code>	<code>int</code>	reviewer	Repair passes performed
<code>trace</code>	<code>list</code>	every node	Ordered, human-readable step log

9.2 Document structure

An RFP and a DPR are different documents. Seven of the DPR's eleven sections have no RFP equivalent, and only the compliance-bearing keys are shared.

RFP — 7 sections	DPR — 11 sections
1 Scope of Work	1 Executive Summary
2 Eligibility & Qualification	2 Project Background & Need Analysis
3 Bid Evaluation Methodology	3 Project Objectives & Scope
4 Payment Schedule	4 Technical Design & Solution Architecture
5 Service Levels (SLA)	5 Implementation Plan & Timeline
6 Data Security & Confidentiality	6 Cost Estimate & Financial Phasing
7 Penalty & Liquidated Damages	7 Operations & Maintenance Service Levels
	8 Data Security & Governance
	9 Penalty & Liquidated Damages
	10 Risk Analysis & Mitigation

RFP — 7 sections	DPR — 11 sections
11 Expected Outcomes & Benefits	

9.3 Finding

```
{
  "rule_id": "PEN-004",
  "pack": "penalty",
  "framework": "Penalty (Liquidated Damages)",
  "severity": "mandatory",
  "message": "The penalty clause must state a rate, an overall cap, ...",
  "citation": "GFR 2017, Rule 175 (enforceability)",
  "fix_hint": "Rewrite with LD rate, cap in rupees, grace period, ..."
}
```

10. Interface Specification

10.1 HTTP endpoints

Method & path	Purpose	Success	Failure
POST /api/run/stream	Run the pipeline, streaming events	200 · text/event-stream	error event on the stream
POST /api/run	Run the pipeline, single response	200 · JSON result	502 on pipeline error
POST /api/export	Download the document	200 · DOCX	423 Locked when gated; 400 if nothing has been run
GET /api/health	Engine, provider, model, tuning, call statistics	200 · JSON	—
GET /api/rulepacks	Active packs and versions	200 · JSON	—
GET /api/structure/{type}	Section outline for RFP or DPR	200 · JSON	—

10.2 Health response

Deliberately diagnostic. It reports not only whether the provider is configured but whether the named model is actually present — the most common cause of a failed first run.

```
{
  "engine": "LangGraph",
  "log_level": "INFO",
  "llm": {
    "provider": "ollama", "model": "qwen2.5:7b", "ok": false,
    "detail": "Model 'qwen2.5:7b' not found in Ollama. Run: ollama pull qwen2.5:7b",
    "stats": { "calls": 26, "errors": 0, "total_seconds": 512.4 }
  },
  "tuning": { "section_retries": 2, "max_review_loops": 3, ... }
}
```

10.3 Exported document

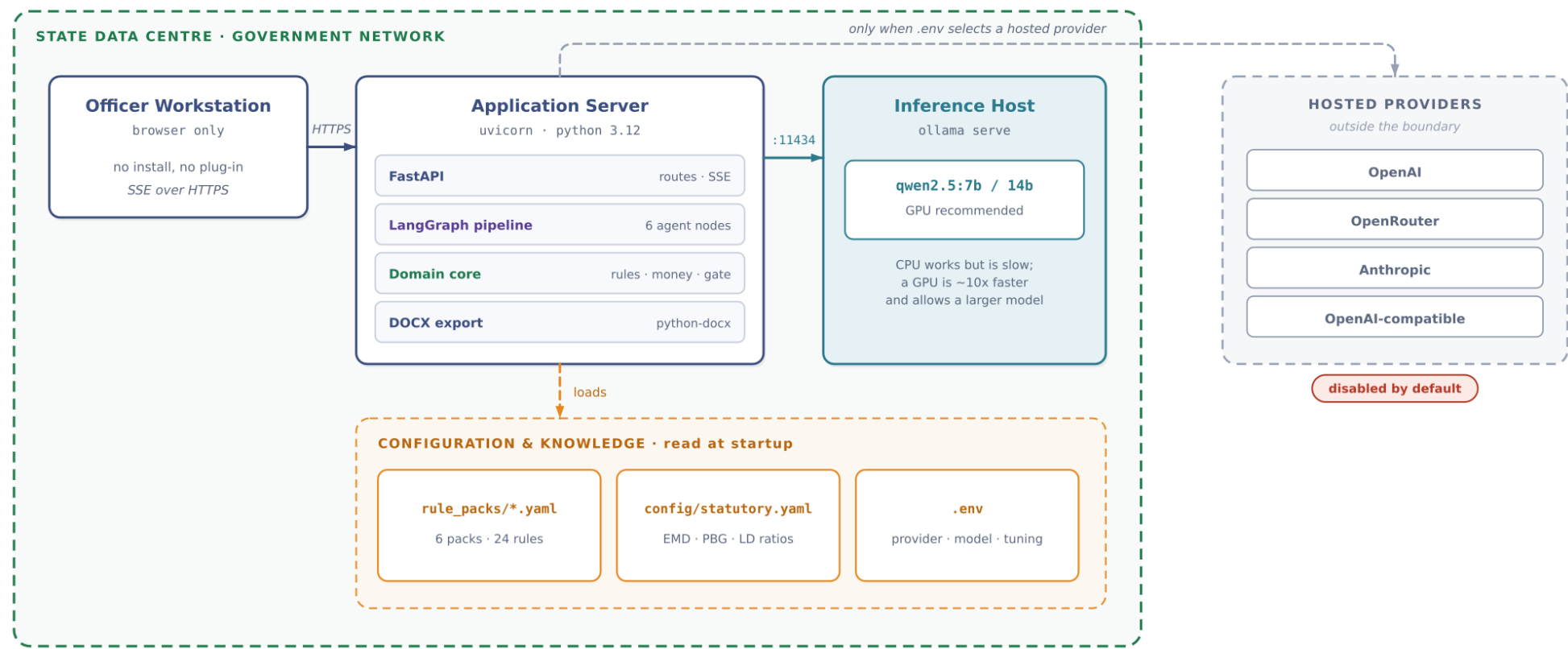
Part	Content
Cover block	Title, document type, estimated cost, duration, compliance percentage
Body	Each section as a heading with paragraphs and bullets
Compliance scorecard	Per-framework table: framework, score, status
Compliance basis	Every statutory reference the document was checked against, so the evidence travels with the file

11. Deployment and Configuration

The default posture is that nothing leaves the government network. A hosted provider can be selected, but it is an explicit act of configuration rather than the path of least resistance.

PraroopAI — Deployment Topology

Default posture: everything inside the government boundary. No project data leaves the network.



Single process serves both the API and the UI — there is no separate front-end build or server.

Figure 6 — Deployment topology. Hosted providers sit outside the trust boundary and are disabled by default.

11.1 Runtime

Component	Requirement	Note
Application server	Python 3.11+, 2 vCPU, 2 GB RAM	Single uvicorn process serving both API and interface
Inference host	Ollama; GPU strongly recommended	May be the same machine. CPU inference works but is roughly ten times slower
Officer workstation	A modern browser	No install, no plug-in, no build step
Storage	Filesystem only	Rule-packs, statutory configuration and .env are read at startup

11.2 Configuration reference

All behaviour is driven from a single .env file. Changing the provider requires no code change and no redeployment.

Setting	Default	Effect
LLM_PROVIDER	ollama	One of ollama · openai · openrouter · anthropic · openai_compatible
LLM_MODEL	qwen2.5:7b	Model name as the provider knows it
LLM_MAX_TOKENS	1600	Upper bound per section. Generation time scales with output length, so this is the most effective speed control
LLM_REQUEST_TIMEOUT	120	Seconds before a call is abandoned
DRAFT_CONCURRENCY	3	Sections drafted in parallel
SECTION_RETRIES	2	Attempts per section before accepting what was returned
MIN_SECTION_CHARS	180	Below this a section is retried
MAX_REVIEW_LOOPS	3	Repair passes before the run completes as-is
LOG_LEVEL	INFO	TRACE · DEBUG · INFO · WARNING · ERROR

11.3 Provider selection

LLM_PROVIDER	Example model	Credential	Data leaves network
ollama	qwen2.5:7b, qwen2.5:14b	none	No
openai	gpt-4o-mini	OPENAI_API_KEY	Yes
openrouter	anthropic/claude-3.5-sonnet	OPENROUTER_API_KEY	Yes
anthropic	claude-3-5-sonnet-20241022	ANTHROPIC_API_KEY	Yes
openai_compatible	any (vLLM, LM Studio)	optional	Depends on endpoint

11.4 Performance characteristics

Drafting dominates the run: one model call per section, eleven sections for a DPR. Measured throughput on a CPU-only Ollama host was approximately ten tokens per second, giving a nine-minute run. The same model on a GPU typically exceeds a hundred tokens per second.

Lever	Effect	Trade-off
GPU inference host	Order-of-magnitude reduction in run time	Requires hardware; by far the largest single improvement
DRAFT_CONCURRENCY = 3	Roughly three times fewer wall-clock rounds	Requires the inference server to accept parallel requests (OLLAMA_NUM_PARALLEL)
LLM_MAX_TOKENS = 700	Shorter sections generate proportionally faster	Less elaborate prose
SECTION_RETRIES = 1	Removes the worst-case second call	A thin section is not retried, but the critic still catches an empty one
Larger model	Markedly better prose and better instruction-following	More VRAM; slower per token, though usually fewer retries

12. Error Handling and Resilience

The design assumes the model will misbehave. Most failure modes are therefore absorbed rather than propagated — the run completes and reports honestly instead of aborting.

Failure	Detection	Response	Visible to officer
Model unreachable	Connection error on the call	Intake falls back to form inputs; drafting records an empty section	Warning in the live log; the critic raises the missing sections
Call times out	Timeout after LLM_REQUEST_TIMEOUT	Logged with the elapsed time and the configured limit; the section is retried once	Live log entry
Unparseable JSON	Extraction fails after fence stripping and object search	Treated as an empty result for that node	Warning in the live log
Model returns an object where prose was expected	Type check	Coerced into labelled bullet lines	None — output is clean
Wrong monetary magnitude	Amount not in the canonical set	Rewritten to the canonical figure	None — output is correct
Model rewrites an unrelated section	Key is not in the finding whitelist	Rejected; the existing section is preserved	Warning in the live log
Model cannot satisfy a mandatory rule	Rule still failing after repair	Clause built deterministically from configuration	Trace shows <i>(rule-built)</i>
Repair loop exhausted	loop_count reaches the bound	Run completes with the gate shut and findings listed	Scorecard shows the open findings; export returns 423
Malformed .env	Byte-level decode with BOM tolerance	Unknown bytes ignored so startup cannot be broken by an editor	None

13. Observability

An operator must be able to answer "what is it doing, and why did it do that" without attaching a debugger.

Level	Shows	When to use
TRACE	Everything, including full prompts and full model replies	A section came out wrong and you need the exact prompt
DEBUG	Every model call with timing and size; every rule evaluation with its pass or fail	Diagnosing behaviour or measuring throughput
INFO	Pipeline milestones, per-section completion, findings, verdict	Normal operation
WARNING	Only problems: empty sections, rejected keys, deterministic fallbacks, exhausted loops	Steady-state monitoring

13.1 Representative output

```
INFO praroopai.agents [draft] start - 11 DPR sections, 3 at a time
INFO praroopai.llm -> draft:1. Executive Summary | ollama/qwen2.5:7b | prompt 1398 chars
INFO praroopai.llm <- draft:1. Executive Summary | 12.4s | 1619 chars
INFO praroopai.agents [draft] 1/11 1. Executive Summary 12.4s 1557 chars
INFO praroopai.agents [compliance] 2 findings (2 mandatory): PEN-002, PEN-004
WARN praroopai.agents [reviewer] rejected 'sla' - not tied to any open finding
WARN praroopai.agents [reviewer] model could not satisfy PEN-002, PEN-004
      - built penalty deterministically from statutory config
INFO praroopai.agents [reviewer] Fixed: penalty (rule-built)
INFO praroopai.agents [compliance] clean - no findings
INFO praroopai.agents [score] overall 100% | mandatory_open=0 | can_finalize=True
```

Three properties are worth noting: each model call is labelled with the section it belongs to; every repair is attributed; and the deterministic fallback announces itself as a warning rather than passing silently.

14. Testing Strategy

Because the trustworthy parts of the system are deterministic, most of it can be tested without a model at all. A mock server emulating the Ollama chat API covers the remainder.

Suite	Covers	Model required
test_pipeline.py	Safe evaluator; derivation arithmetic; milestone totals; sanitiser behaviour; deterministic fallbacks; the hard gate; DPR completeness rules; citation propagation	No
test_providers.py	Each provider adapter builds the correct request and parses the correct response shape, including Anthropic's separate system field	No — httpx is stubbed
test_logging.py	Log level configuration, the custom TRACE level, and the progress bus	No
run_e2e.py	A full run for both document types against a mock model	Mock only

14.1 Adversarial end-to-end test

The end-to-end test does not use a well-behaved mock. It uses one that reproduces the failure modes observed in practice: it writes markdown headings and bold markers, states monetary amounts a hundred times too large, returns an unrelated section key when asked to repair, and refuses to produce a compliant penalty clause however it is asked.

The test then asserts that the document still reaches 100%, that no markdown survives, and that no incorrect amount appears — which is a direct test of the design principle rather than of any single function.

15. Limitations and Roadmap

15.1 Known limitations

Limitation	Consequence	Mitigation
Prose quality tracks model size	A 1.5B model produces thin sections	Correctness is guaranteed regardless; a 7B or 14B model is recommended for readable output
Rule-packs are illustrative	Twenty-four rules do not cover the whole of GFR	The format and the engine are complete; deepening the packs is authoring work, not development
No persistence	A run exists only in memory	A repository and version history are the natural next increment
Single-user pilot	No accounts, no concurrent sessions	Deliberate for the pilot; the pipeline itself is stateless per run
Fallback clauses are generic	Rule-built text is correct but not tailored	It is attributed as rule-built so an officer can elaborate it knowingly

15.2 Roadmap

Increment	Description
1	Deepen the rule-packs with departmental counsel and add the remaining GFR provisions
2	Persist runs; allow an officer to reopen and revise a draft
3	Officer accounts and an audit log of waivers, so a deliberate override is recorded rather than silent
4	Bilingual output (Hindi and English) using the same rule engine
5	Departmental templates layered above the base document structures

PraroopAI — grounded, rule-anchored, on-premise. Prepared for UKIS 2026, ITDA, Government of Uttarakhand.